

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Navya Maheshwari – 24011101071

AI & DS – Batch 'B1'

Experiment 3

Design and Implementation of a Convolutional Neural Network for Image Classification

1. Objective

The objective of this experiment is to design and implement a Convolutional Neural Network (CNN) for image classification using the TensorFlow/Keras framework. It covers the fundamental building blocks of CNNs (convolutional layers, activation functions, pooling layers, flattening, and fully connected layers) and how convolution kernels extract visual features such as edges, textures, and patterns. The experiment also studies the influence of key hyperparameters (kernel size, stride, padding, number of filters, optimizer, batch size, epochs), visualizes intermediate feature maps, compares pooling strategies, analyzes trainable parameters, and evaluates a trained CNN on the CIFAR-10 dataset using standard performance metrics.

2. Background Theory

A Convolutional Neural Network is a deep neural network specialized for grid-like data such as images. Instead of connecting every input pixel to every neuron, a CNN applies small learnable filters (kernels) that slide across the image, using spatial locality and weight sharing to reduce the number of parameters while still detecting local patterns. Stacked convolutional layers learn a hierarchy of features: early layers detect edges and simple textures, and deeper layers combine these into more abstract object parts.

Convolution Operation

For a 2D input I and a kernel K of size $k \times k$, the convolution (cross-correlation) output at position (i, j) is:

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n) + b$$

where b is a bias term. Each filter produces one feature map.

Output Spatial Size

For an input of size W , kernel size F , padding P , and stride S , the output dimension along that axis is:

$$O = \left\lfloor \frac{W - F + 2P}{S} \right\rfloor + 1$$

Trainable Parameters of a Convolutional Layer

For a convolutional layer with F_{in} input channels, F_{out} filters, and kernel size $k \times k$, the number of trainable parameters is:

$$\text{Params} = (k \times k \times F_{in} + 1) \times F_{out}$$

where the +1 accounts for the per-filter bias.

Pooling

Pooling downsamples feature maps to reduce spatial dimensions and computational cost while adding some translation invariance. Max pooling takes the maximum value in each window (retaining the strongest activation), while average pooling takes the mean (a smoother, more global summary).

3. CNN Building Blocks

Component	Role
Convolutional Layer	Applies learnable kernels to extract local features (edges, textures, patterns).
Activation (ReLU)	Introduces non-linearity; passes positive activations and zeroes negatives.
Pooling Layer	Reduces spatial size of feature maps; improves efficiency and invariance.
Flatten	Converts the final 2D feature maps into a 1D vector for the dense layers.
Fully Connected	Combines extracted features to produce class scores.
Softmax	Converts final scores into a probability distribution over the ten classes.
Dropout	Randomly deactivates units during training to reduce overfitting.

Activation Functions

Activation	Output Range	Typical Applications
ReLU	$[0, \infty)$	Hidden layers in CNNs
Leaky ReLU	$(-\infty, \infty)$	Avoiding dying ReLU
Sigmoid	$(0, 1)$	Binary classification output
Tanh	$(-1, 1)$	Hidden layers (traditional networks)
Softmax	$(0, 1)$	Multi-class classification output layer

Note: This experiment uses ReLU in the hidden layers and Softmax in the output layer.

4. Dataset Description

Dataset: CIFAR-10 **Source:** Canadian Institute For Advanced Research (via `keras.datasets`)
<https://www.cs.toronto.edu/~kriz/cifar.html>

Total Images	60,000
Training / Test	50,000 / 10,000
Image Size	$32 \times 32 \times 3$ (RGB)
Classes	10
Missing Values	None
Task	Multi-class Image Classification

Classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck. The ten classes are mutually exclusive and balanced (6,000 images per class).

5. Experimental Procedure

Task 1: Dataset Exploration — Load CIFAR-10, inspect array shapes, display the class distribution and a grid of sample images with labels.

Task 2: Data Preprocessing — Normalize pixel values to $[0, 1]$ and one-hot encode the class labels.

Task 3: CNN Architecture Design — Build a CNN with stacked convolution–activation–pooling blocks followed by flatten and fully connected layers; inspect the trainable-parameter summary.

Task 4: Model Training — Compile with an optimizer and categorical cross-entropy loss; train for a fixed number of epochs with a chosen batch size, recording training and validation accuracy/loss.

Task 5: Feature Map Visualization — Extract and visualize intermediate feature maps from convolutional layers to observe learned hierarchical representations.

Task 6: Model Evaluation — Compute accuracy, precision, recall, F1-score, confusion matrix, and the full classification report on the test set.

Task 7: Pooling Strategy Comparison — Retrain an identical architecture using average pooling in place of max pooling and compare validation behaviour.

6. Source Code

<https://github.com/nzoopxw/deep-learning-lab-sem5/tree/main/Lab-03>

7. Experimental Results

Dataset Overview

The training set has shape $50000 \times 32 \times 32 \times 3$ and the test set $10000 \times 32 \times 32 \times 3$, with no missing values. All ten classes are equally represented with 5,000 training images each.

Task-wise Outputs

Task 1: Dataset Exploration. The class distribution and a grid of labelled sample images were displayed.



Figure 1: Class distribution of the CIFAR-10 training set (balanced across all ten classes).



Figure 2: Sample images from the CIFAR-10 training set with their class labels.

Task 3: CNN Architecture. The model summary below reports the output shape and trainable parameters of each layer. The full network contains **402,986** trainable parameters.

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 32)	896
conv2d_1 (Conv2D)	(None, 32, 32, 32)	9,248
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
dropout (Dropout)	(None, 16, 16, 32)	0
conv2d_2 (Conv2D)	(None, 16, 16, 64)	18,496
conv2d_3 (Conv2D)	(None, 16, 16, 64)	36,928
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0
dropout_1 (Dropout)	(None, 8, 8, 64)	0
conv2d_4 (Conv2D)	(None, 8, 8, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 128)	0
dropout_2 (Dropout)	(None, 4, 4, 128)	0
flatten (Flatten)	(None, 2048)	0
dense (Dense)	(None, 128)	262,272
dropout_3 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

Total params: 402,986 (1.54 MB)
Trainable params: 402,986 (1.54 MB)
Non-trainable params: 0 (0.00 B)

Figure 3: Model architecture and per-layer trainable parameters (`model.summary()`).

Task 4: Model Training. The network was trained for 100 epochs with the Adam optimizer and a batch size of 64, printing per-epoch training and validation accuracy and loss.

```

782/782 - 6s - 7ms/step - accuracy: 0.8646 - loss: 0.3903 - val_accuracy: 0.8363 - val_loss: 0.5369
Epoch 80/100
782/782 - 5s - 7ms/step - accuracy: 0.8612 - loss: 0.4033 - val_accuracy: 0.8306 - val_loss: 0.5567
Epoch 81/100
782/782 - 6s - 7ms/step - accuracy: 0.8640 - loss: 0.3933 - val_accuracy: 0.8268 - val_loss: 0.5719
Epoch 82/100
782/782 - 5s - 7ms/step - accuracy: 0.8654 - loss: 0.3922 - val_accuracy: 0.8284 - val_loss: 0.5655
Epoch 83/100
782/782 - 6s - 8ms/step - accuracy: 0.8638 - loss: 0.3996 - val_accuracy: 0.8320 - val_loss: 0.5565
Epoch 84/100
782/782 - 5s - 7ms/step - accuracy: 0.8651 - loss: 0.3962 - val_accuracy: 0.8336 - val_loss: 0.5321
Epoch 85/100
782/782 - 6s - 7ms/step - accuracy: 0.8636 - loss: 0.3975 - val_accuracy: 0.8353 - val_loss: 0.5305
Epoch 86/100
782/782 - 6s - 7ms/step - accuracy: 0.8672 - loss: 0.3885 - val_accuracy: 0.8344 - val_loss: 0.5445
Epoch 87/100
782/782 - 5s - 7ms/step - accuracy: 0.8649 - loss: 0.3936 - val_accuracy: 0.8335 - val_loss: 0.5454
Epoch 88/100
782/782 - 6s - 7ms/step - accuracy: 0.8642 - loss: 0.3944 - val_accuracy: 0.8346 - val_loss: 0.5383
Epoch 89/100
782/782 - 5s - 7ms/step - accuracy: 0.8647 - loss: 0.3984 - val_accuracy: 0.8250 - val_loss: 0.5751
Epoch 90/100
782/782 - 6s - 7ms/step - accuracy: 0.8678 - loss: 0.3899 - val_accuracy: 0.8343 - val_loss: 0.5427
Epoch 91/100
782/782 - 5s - 7ms/step - accuracy: 0.8669 - loss: 0.3918 - val_accuracy: 0.8298 - val_loss: 0.5646
Epoch 92/100
782/782 - 6s - 7ms/step - accuracy: 0.8675 - loss: 0.3863 - val_accuracy: 0.8368 - val_loss: 0.5329
Epoch 93/100
782/782 - 6s - 7ms/step - accuracy: 0.8664 - loss: 0.3933 - val_accuracy: 0.8283 - val_loss: 0.5597
Epoch 94/100
782/782 - 10s - 13ms/step - accuracy: 0.8659 - loss: 0.3949 - val_accuracy: 0.8359 - val_loss: 0.5268
Epoch 95/100
782/782 - 5s - 7ms/step - accuracy: 0.8661 - loss: 0.3932 - val_accuracy: 0.8337 - val_loss: 0.5335
Epoch 96/100
782/782 - 10s - 13ms/step - accuracy: 0.8644 - loss: 0.3922 - val_accuracy: 0.8314 - val_loss: 0.5458
Epoch 97/100
782/782 - 6s - 7ms/step - accuracy: 0.8662 - loss: 0.3907 - val_accuracy: 0.8333 - val_loss: 0.5616
Epoch 98/100
782/782 - 5s - 7ms/step - accuracy: 0.8674 - loss: 0.3822 - val_accuracy: 0.8335 - val_loss: 0.5551
Epoch 99/100
782/782 - 6s - 7ms/step - accuracy: 0.8672 - loss: 0.3880 - val_accuracy: 0.8344 - val_loss: 0.5531
Epoch 100/100
782/782 - 6s - 7ms/step - accuracy: 0.8684 - loss: 0.3895 - val_accuracy: 0.8361 - val_loss: 0.5504

```

Figure 4: Epoch-wise training log (accuracy and loss on train/validation sets).

Task 6: Model Evaluation. The trained model was evaluated on the 10,000-image test set, achieving a test accuracy of **0.8361**.

313/313 ————— 1s 4ms/step

Test Accuracy : 0.8361
Precision (macro): 0.8384534334507405
Recall (macro) : 0.8360999999999998
F1-score (macro) : 0.8362122107005756

Classification Report:				
	precision	recall	f1-score	support
airplane	0.86	0.82	0.84	1000
automobile	0.93	0.91	0.92	1000
bird	0.77	0.78	0.77	1000
cat	0.68	0.72	0.70	1000
deer	0.86	0.80	0.83	1000
dog	0.78	0.76	0.77	1000
frog	0.82	0.91	0.86	1000
horse	0.93	0.81	0.87	1000
ship	0.90	0.92	0.91	1000
truck	0.85	0.93	0.89	1000
accuracy			0.84	10000
macro avg	0.84	0.84	0.84	10000
weighted avg	0.84	0.84	0.84	10000

Figure 5: Accuracy, precision, recall, F1-score, and classification report.

8. Generated Plots with Interpretation

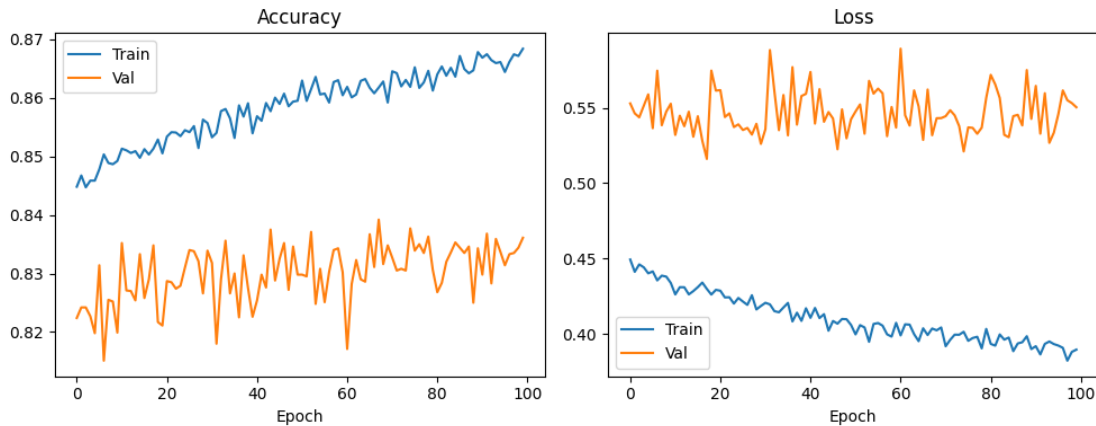


Figure 6: Training and Validation Accuracy/Loss Curves (100 epochs).

Interpretation: Training accuracy rises steadily (roughly 0.845 to 0.868) and training loss falls smoothly, so the model keeps fitting the training data. Validation accuracy plateaus around 0.82–0.84 and validation loss stays roughly flat near 0.55 instead of continuing to fall. The gap between the rising training curve and the flat validation curve indicates mild overfitting in the later epochs; dropout limits it, but the model stops generalizing better after about epoch 30–40.

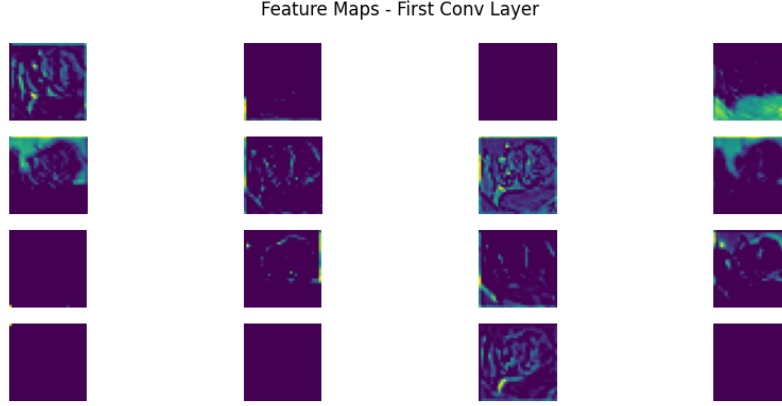


Figure 7: Feature Maps from the First Convolutional Layer.

Interpretation: The first-layer feature maps respond to low-level features: some channels highlight edges and outlines, others activate on broad colour or brightness regions, and a few are nearly inactive for this input. The first convolutional layer therefore learns simple, general-purpose detectors rather than class-specific ones, which is the expected first stage of the feature hierarchy.

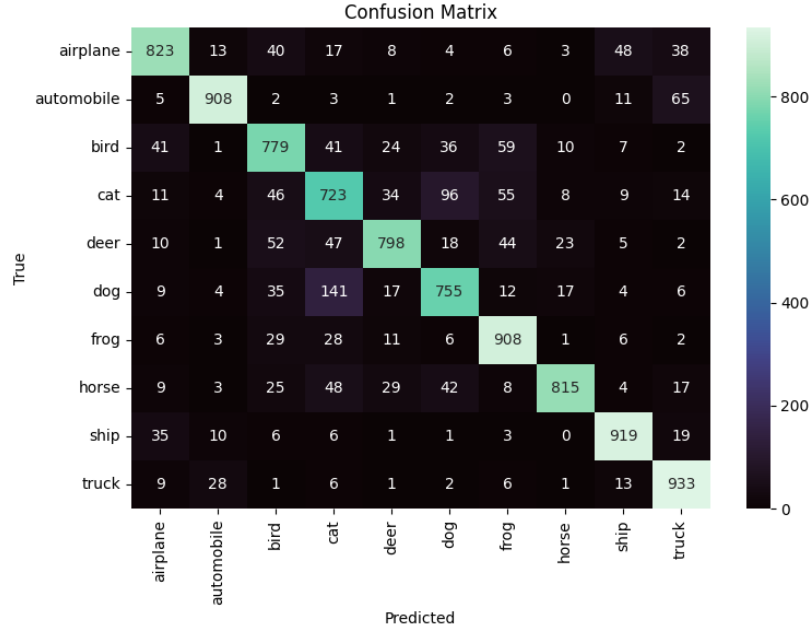


Figure 8: Confusion Matrix on the Test Set (Max Pooling model).

Interpretation: The strong diagonal shows most classes are classified correctly. Vehicle classes are the most reliable (truck 933, ship 919, automobile 908, frog 908), while the animal classes account for most of the confusion. The largest off-diagonal entry is dog→cat (141), with cat→dog (96) also high, reflecting the similar shape and texture of the two animals at 32×32 resolution. Bird, deer, and horse are also mixed with other animals. This matches the classes' shared low-level appearance rather than any labelling issue.

9. Performance Tables

Training Summary

Dataset	CIFAR-10 (60,000 images)
Train/Test Split	50,000 / 10,000
Input Shape	$32 \times 32 \times 3$
Optimizer	Adam
Loss Function	Categorical Cross-Entropy
Batch Size	64
Epochs	100
Total Trainable Parameters	402,986
Test Accuracy	0.8361
Precision (macro)	0.8385
Recall (macro)	0.8361
F1-score (macro)	0.8362

Per-Class Performance

Class	Precision	Recall	F1-score	Support
airplane	0.86	0.82	0.84	1000
automobile	0.93	0.91	0.92	1000
bird	0.77	0.78	0.77	1000
cat	0.68	0.72	0.70	1000
deer	0.86	0.80	0.83	1000
dog	0.78	0.76	0.77	1000
frog	0.82	0.91	0.86	1000
horse	0.93	0.81	0.87	1000
ship	0.90	0.92	0.91	1000
truck	0.85	0.93	0.89	1000
Macro avg	0.84	0.84	0.84	10000

The per-class numbers match the confusion matrix: `cat` is the weakest class (F1 0.70) and `automobile` the strongest (F1 0.92), with the vehicle classes generally outperforming the animal classes.

Trainable Parameter Analysis

Applying $\text{Params} = (k \times k \times F_{in} + 1) \times F_{out}$ to each convolutional layer reproduces the Keras counts exactly:

Layer	Kernel	F_{in}	F_{out}	Params
conv2d	3×3	3	32	896
conv2d_1	3×3	32	32	9,248
conv2d_2	3×3	32	64	18,496
conv2d_3	3×3	64	64	36,928
conv2d_4	3×3	64	128	73,856
Total convolutional				139,424
Dense (262,272 + 1,290)				263,562
Total trainable				402,986

For the first layer, $(3 \times 3 \times 3 + 1) \times 32 = 28 \times 32 = 896$, matching Keras. The five convolutional layers together hold about 35% of the parameters (139,424), while the single **Dense**(128) layer after **Flatten** holds 262,272 on its own (about 65% of the model). Most of the weight budget therefore sits in the fully connected head rather than the convolutional stack.

10. Pooling Strategy Comparison

To study the effect of the pooling operation, an identical architecture was retrained with **AveragePooling2D** in place of every **MaxPooling2D** layer. All other settings (optimizer, batch size, 100 epochs, dropout) were kept constant, so any difference is due to the pooling choice.

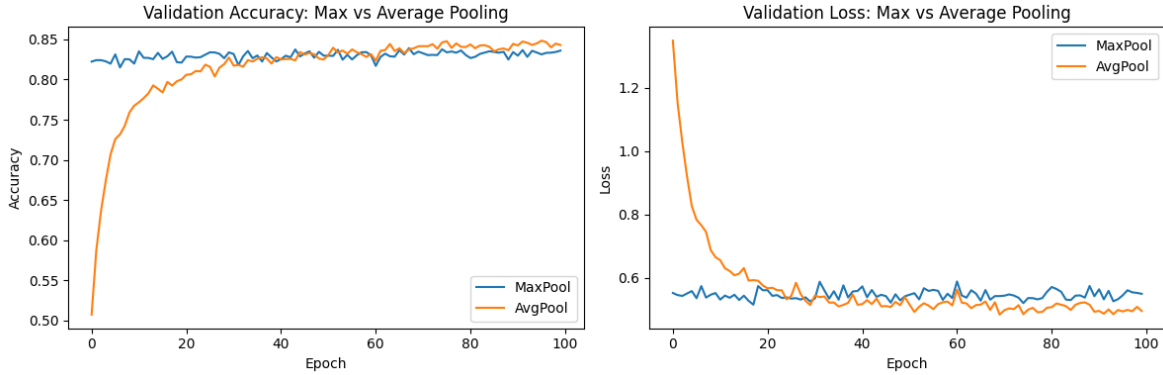


Figure 9: Validation accuracy and loss: Max pooling vs. Average pooling.

Pooling	Final Train Acc	Final Val Acc	Final Val Loss
Max Pooling	0.8684	0.8361	0.5504
Average Pooling	0.8839	0.8431	0.4963

Interpretation: Average pooling slightly outperformed max pooling on this architecture, with higher final validation accuracy (0.8431 vs. 0.8361) and lower validation loss (0.4963 vs. 0.5504). The plot also shows a difference in early training: the average-pooling model starts much lower and climbs over the first 25 epochs, while the max-pooling model reaches its plateau almost immediately.

After convergence the two track closely, with average pooling keeping a small edge. One likely reason is that average pooling’s smoother, whole-window summaries retain more of the background and low-contrast information in these small 32×32 images. This indicates the choice of pooling is data-dependent rather than fixed.

11. Discussion (Additional Tasks)

1. Effect of kernel size, stride, and padding.

Small kernels (3×3 , as used here) capture fine local detail and, when stacked, reach a large effective receptive field with fewer parameters than one large kernel. A larger stride downsamples more aggressively, shrinking the feature maps and cutting computation at the cost of spatial detail. “Same” padding preserves the spatial dimensions through a convolution (here the 32×32 resolution is held until pooling reduces it), while “valid” padding shrinks them, so padding controls how much border information is kept across layers.

2. Compare max pooling and average pooling.

Aspect	Max Pooling	Average Pooling
Operation	Takes maximum in each window	Takes mean of each window
Effect	Retains strongest activation	Produces smoother summary
Best for	Sharp features, edges, textures	Global, low-contrast information

Max pooling keeps the strongest activation in each window and is the more common default; average pooling produces smoother feature maps. In this experiment (Section 10) average pooling gave slightly better validation accuracy and loss, so the preferred strategy is data-dependent.

3. Effect of number of filters.

More filters let each convolutional layer learn a richer set of feature detectors, which generally improves accuracy up to a point. The model widens the filters progressively ($32 \rightarrow 64 \rightarrow 128$) so that deeper layers, working on smaller spatial maps, can represent more abstract patterns. Past a point, extra filters increase parameters, memory, and training time with little gain and can encourage overfitting on a moderate-sized dataset like CIFAR-10.

4. Effect of optimizer, batch size, and epochs.

The optimizer sets how weights are updated; adaptive methods such as Adam (used here) usually converge faster than plain SGD. Larger batch sizes give smoother, more stable gradients but need more memory and can generalize slightly worse, while smaller batches add useful gradient noise. Too few epochs underfit and too many overfit once validation loss stops improving, which is visible in this run: validation accuracy plateaued around epoch 30–40 while training accuracy kept rising. This is where dropout and early stopping matter.

5. How does a CNN learn hierarchical representations?

Early convolutional layers work directly on pixels and learn low-level features such as edges and colour gradients, seen in the first-layer feature maps of Section 8. Later layers combine these

into textures and object parts, and the deepest layers assemble parts into whole objects. Pooling between blocks enlarges the receptive field so that later neurons cover larger regions of the image, moving from simple to abstract features.

12. Conclusion

A CNN was designed and trained on CIFAR-10 for ten-class image classification using TensorFlow/Keras. The architecture combined stacked convolution-ReLU-pooling blocks with dropout and fully connected layers (402,986 trainable parameters) and, trained for 100 epochs with Adam, reached a test accuracy of **0.8361** with a macro F1-score of 0.8362. Feature-map visualizations showed that early layers respond to edges and simple textures, confirming the hierarchical nature of the learned representations. The confusion matrix showed the animal classes, especially cat and dog, were most often confused, matching their shared low-level appearance, while vehicle classes were classified most reliably. The parameter analysis showed most of the model’s weights sit in the dense head rather than the convolutional stack. A controlled comparison of pooling strategies found that average pooling slightly outperformed max pooling here, showing the choice of pooling is data-dependent. The experiment gave practical experience in CNN design, feature extraction, training, evaluation, and hyperparameter analysis for computer-vision tasks.

13. References

1. Y. LeCun, L. Bottou, Y. Bengio and P. Haffner, “Gradient-Based Learning Applied to Document Recognition,” *Proceedings of the IEEE*, 1998.
2. A. Krizhevsky, “Learning Multiple Layers of Features from Tiny Images,” Technical Report, University of Toronto, 2009.
3. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
4. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
5. F. Chollet, *Deep Learning with Python*, Manning, 2021.
6. TensorFlow / Keras Documentation.